

Take-Home Angular Design System Exercise

Senior Angular Software Developer III

Context

Caseware builds software used by accounting professionals to manage engagement work. The screens that work is done on are assembled from a shared Angular UI kit used by several product teams, so the kit's component APIs, accessibility behaviour and theming are the parts that carry the most leverage — and the most risk.

This exercise focuses on design system engineering at a Senior level: component API design, accessibility, token and theming architecture, and the judgment involved in evolving a shared library other teams already depend on.

The Problem

A supplied UI kit contains one component and a token layer that stops at raw values. You will add the component the review screens are missing, establish the token layer that makes the kit themeable, and deal with the component that is already there.

Some of the design is intentionally underspecified. Where the brief states a required behaviour, treat it as required. Where it leaves a decision open, identify the decisions that matter, make reasonable choices, and apply them consistently across the kit.

What We Provide

Alongside this document you will receive a starter project so that you can focus on the design system problem rather than on setup:

- ☐ An Angular 21 application, zoneless, with strict TypeScript and Vitest configured
- ☐ A UI kit under `src/lib/` with a single public entry point
- ☐ A supplied primitive token layer, and a stub for the layer above it
- ☐ One supplied component, `cw-status-badge`
- ☐ A workbench application that consumes the kit, with sample reviewer, change-group and engagement data
- ☐ A README describing the starter, the kit and the token layer

For clarity, the following is fixed versus intentionally left to you:

- ☐ **Fixed:** the primitive token file, the required component behaviours listed below, the kit having a single public entry point, and the scope boundaries in `_Assumptions & Constraints_`
- ☐ **Open:** every public API you design, the ARIA and interaction pattern you choose, the semantic token layer and how a theme overrides it, how styles are structured, what your documentation surface is, and what you test

Part 1: The Reviewer Picker (Primary Focus)

Build a single-select component for the kit, so a user can choose which reviewer an engagement is assigned to. The workbench has a `_Review filters_` section with a form control already in place and a placeholder where it belongs.

It is a kit component, not a screen: another team should be able to use it for something that is not reviewers at all.

The following behaviours are required:

- ☐ It is labelled, shows the current selection, and opens a list of the available options
- ☐ Choosing an option updates the value and closes the list
- ☐ It integrates with Angular forms — reactive or template-driven, your choice — including value, disabled state, and touched/dirty
- ☐ It is fully operable using the keyboard alone: opening, moving through the options, choosing one, and dismissing the list without changing the selection
- ☐ Options that cannot be chosen are conveyed as unavailable, not merely styled as unavailable
- ☐ Assistive technology can determine what the control is, what its current value is, that a list has opened, and which option is currently active
- ☐ Focus is never lost to the document, and dismissing the list returns focus somewhere sensible
- ☐ It behaves correctly with the supplied fixture data and would still behave correctly with several hundred options

How you achieve these is yours. In particular, the ARIA and focus pattern you adopt, whether options are passed in or projected, whether typing filters or jumps, and how the popup is positioned are all decisions we expect you to make deliberately rather than answers we are looking for.

Part 2: Tokens, Theming, and the Component You Inherited

Establish the semantic token layer. `_primitives.scss` is supplied and `raw._semantic.scss` is a stub. Design the layer between them so that kit components consume meaning rather than values.

Deliver a second theme. Alongside the default, provide one alternative theme — dark or high-contrast, your choice — that every kit component responds to without duplicating or forking component styles. Make it possible to see both themes in the workbench.

Deal with `cw-status-badge`. It is real code of the kind that accumulates in a shared library before anyone owns it, and it has problems in its public API, its accessibility, and its styling. Identify them, fix the ones that matter, and update the consumers in the workbench. If you change its public API, treat that as a breaking change to a library other teams use, and say so in your notes.

You do not have to fix everything you find. We are more interested in what you judged worth changing, what you deliberately left, and whether you can tell the difference.

Part 3: Focused Tests & Decision Notes

Add a small set of focused tests for the behaviour you believe carries the highest correctness risk. We are more interested in what you chose to test and why than in test count or coverage percentage. For a component like this, consider where a regression would actually reach a user.

Include a short **DECISIONS.md**. Briefly describe 3–5 important decisions or assumptions and why. Focus on places where another reasonable implementation could have behaved differently, and include at least one alternative you considered and what would have changed if you had chosen it.

Part 4: Adoption Note

Include a short **ADOPTION.md** — half a page is plenty, and prose is fine. Cover:

- ☐ How you would version and release the change you made to `cw-status-badge`, and what consuming teams would have to do
- ☐ What you would document or automate so that adopting it is safe rather than merely possible
- ☐ What you would expect to go wrong if two different versions of this kit ended up loaded in the same page, as can happen with module federation, and what about your design makes that better or worse

This part is written, not implemented. We are looking for how you think about the kit as a shipped artifact, not for an implementation of packaging.

AI Usage (Important)

You may use any AI tools you normally use (e.g., ChatGPT, Copilot, Claude). We have scoped the exercise assuming such tools are available; using them is not required.

During the follow-up review, we will ask you to explain:

- ☐ Where AI helped you
- ☐ Where you corrected, rewrote, or ignored AI output
- ☐ How you verified important generated code or reasoning
- ☐ The important engineering decisions behind the submitted solution

Accessibility behaviour is an area where generated code is often confidently wrong. We are evaluating judgment and ownership, not whether you avoided AI. Using AI heavily is not a mark against you; submitting something you cannot account for is.

Assumptions & Constraints

- ☐ You may state reasonable assumptions; record material ones in **DECISIONS.md**
- ☐ Treat `_primitives.scss` as fixed, though you may add a primitive if one is genuinely missing
- ☐ Single-select only. Multi-select, asynchronous option loading, virtual scrolling, and popup collision or flip handling are explicitly out of scope
- ☐ Do not spend significant time on Storybook setup, build or packaging configuration, publishing, responsive layout, animation, or advanced visual design

- ☐ Basic styling is sufficient; visual polish is not a scoring focus, but legibility and contrast are part of accessibility and do count
- ☐ You may add dependencies, but be prepared to justify each one. Reaching for an existing component library for Part 1 defeats the exercise
- ☐ Optimize for clarity, correctness, and coherent tradeoffs over completeness

Deliverables

Please submit:

- ☐ The Angular project with your implementation
- ☐ Your focused tests
- ☐ DECISIONS.md
- ☐ ADOPTION.md
- ☐ Submission notes covering AI usage, approximate time spent, what you would do next if you had more time, and one risk or limitation you knowingly left in the solution together with the next test you would write for it

What We're Evaluating

- ☐ Component API design: whether the public surface is clear, hard to misuse, and reusable beyond the case in front of you
- ☐ Accessibility: semantics, keyboard operation, focus management, and what assistive technology actually receives
- ☐ Token and theming architecture: whether the layer you built makes a second theme cheap and keeps component styles out of it
- ☐ Judgment on inherited code: what you changed, what you left, and how you communicated a breaking change
- ☐ Testing choices, and whether they target meaningful risk
- ☐ Angular and TypeScript implementation quality, and use of current framework idiom
- ☐ Communication: decision notes, adoption note, and coherence between them and the code
- ☐ Thoughtful use of AI in the development process, and ownership of the submitted result

Time Expectation

We expect this exercise to take approximately 3 hours total. Please prioritize a coherent working solution and clear decisions over completeness or polish. Spending significantly more time is not expected and is not considered an advantage.

If you run out of time, sacrifice in this order:

- ☐ Visual polish and non-essential styling
- ☐ Breadth of tests
- ☐ The depth of the `cw-status-badge` rework, beyond its most important problems

Please do not sacrifice the required behaviours of the reviewer picker, the accessibility of what you do build, or the coherence between your token layer and your components.

Please tell us approximately how much time you spent so we can calibrate the exercise; time spent is not a scoring signal. We would rather review a smaller, coherent submission than an exhaustive one. If you ran short, say so and tell us what you would do next and why.

Please do not over-optimize.

Follow-Up

We will review your submission. If we like it, you will be invited to a live technical review to discuss the solution. We will walk through the submitted code and tests, discuss important decisions and tradeoffs, operate the component by keyboard together, and may introduce a small change in requirements and ask you to explain how you would adapt the design.

Please be prepared to explain, defend, and reconsider the important decisions in anything you submit, including code produced with the assistance of AI tools.